

图 实验报告

姓名：时分

学号：PB23151799

设计目标

在C++环境下，利用图的相关操作，完成如下程序设计：

1. 图的遍历

输入一个无向图，输出图的深度优先搜索遍历顺序与广度优先搜索遍历顺序（要求：当有多个节点可以搜索时，优先去节点编号最小的那个）。

输入格式：第一行两个数 n, m ($1 < n < 30, 1 < m < 300$)，分别表示顶点数量和边的数量。接下来的 m 行每行输入两个数 a, b ，表示顶点 a 与顶点 b 之间有边相连，顶点编号从 1 到 n 。最后输入一个数 s 表示遍历的起始顶点编号。

输出格式：输出两行序列，第一行为深度优先搜索遍历的顺序，第二行为广度优先搜索遍历的顺序。

2. 求通讯网的最小代价生成树

输入一个无向通讯网图，用 Prim 和 Kruskal 算法计算最小生成树并输出。

输入格式：第一行是两个数 n, m ($1 < n < 10000, 1 < m < 100000$)，分别表示顶点数量和边的数量。接下来的 m 行每行输入三个数 a, b, w ；表示顶点 a 与顶点 b 之间有代价为 w 的边相连，顶点编号从 1 到 n 。

输出格式：一个数，即最小生成树的各边的长度之和。

3. 铁路交通网的最短路径

输入一个无向铁路交通图、始发站和终点站，用 Dijkstra 算法计算从始发站到终点站的最短路径。

输入格式：第一行是两个数 n, m ($1 < n < 100000, 1 < m < 1000000$)，分别表示顶点数量和边的数量。接下来的 m 行每行输入三个数 a, b, w ；表示顶点 a 与顶点 b 之间有长度为 w 的边相连，顶点编号从 1 到 n 。最后输入两个数 s, t 表示遍历的起始顶点编号和终点编号。

输出格式：一个数，为从起点到终点的最短路径长度。

4. 显示图

可以直接使用题目 1 的输入数据画出图的情况则为满足要求，要求图整体美观，可以体现出顶点的编号，边的交叉尽量少。

图的遍历

算法设计

深度优先遍历（DFS）采用递归算法，算法思想如下：

1. 初始化访问情况数组，从起始节点开始，将其标记为已访问，以避免重复访问。
2. 对于起始节点的每一个未访问的邻接节点，递归地调用深度优先遍历函数。这意味着每次找到一个新的邻接节点，就将其视为新的起始节点，继续深入探索它的邻接节点。
3. 当一个节点的所有邻接节点都被访问后，递归函数会返回上一层调用。回溯到上一层后，继续探索上一层节点的其他未访问邻接节点。

4. 当所有节点都被访问过，或者没有更多的邻接节点可以探索时，递归过程结束，从小到大对未访问结点再次进行如上操作。

广度优先遍历（BFS）借助队列实现，算法思想如下：

1. 初始化访问情况数组，从起始节点开始，将其标记为已访问，以避免重复访问。
2. 访问起始结点并将起始节点入队。
3. 直到队列为空为止循环：取队列头节点，从小到大访问其未访问的邻接节点，并将它们入队。
4. 循环结束，从小到大对未访问结点再次进行如上操作。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

在BFS中需要用到队列，链队列结点与队列类型定义如下：

```
typedef struct QNode{
    int data;
    struct QNode *next;
}QNode;

typedef struct Queue{
    QNode *front;
    QNode *rear;
}LinkQueue;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
Status EnQueue(LinkQueue &Q, int data)
    //入队操作
Status DeQueue(LinkQueue &Q, int &data)
    //出队操作
Status QueueEmpty(LinkQueue Q)
    //判断队列是否为空
Status DFSTraverse(MGraph G, int start)
    //深度优先搜索
void DFS(MGraph G, int u, int *visited)
    //深度优先搜索的递归算法
Status BFSTraverse(MGraph G, int start)
    //广度优先搜索
void BFS(MGraph G, int u, int *visited, LinkQueue &Q)
    //广度优先搜索利用队列的循环算法
```

注意本题需要从输入的起点出发遍历，故在搜索中先调用起点的DFS和BFS，再从顶点1开始对未访问的顶点依次调用。

调试分析

测试数据Input.txt采用输入样例数据：

```
5 6
1 5
1 4
3 1
2 1
2 5
5 3
1
```

未发现明显问题。

算法的时空分析

各函数时空复杂度如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度：O(arcnum)
    //空间复杂度：O(1)
Status EnQueue(LinkQueue &Q,int data)
    //入队操作
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status DeQueue(LinkQueue &Q,int &data)
    //出队操作
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status QueueEmpty(LinkQueue Q)
    //判断队列是否为空
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status DFSTraverse(MGraph G,int start)
    //深度优先搜索
void DFS(MGraph G,int u,int *visited)
    //深度优先搜索的递归算法
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(1)
Status BFSTraverse(MGraph G,int start)
    //广度优先搜索
void BFS(MGraph G,int u,int *visited,LinkQueue &Q)
    //广度优先搜索利用队列的循环算法
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(1)
```

测试结果及分析

输入内容：

```
5 6
1 5
1 4
3 1
2 1
2 5
5 3
1
```

显示器输出：

```
1 2 5 3 4
1 2 3 4 5
```

求通讯网的最小代价生成树

算法设计

Prim算法的基本思想是从一个起始顶点开始，逐步扩展生成树，直到包含所有顶点，算法思想如下：

1. 从连通无向网G中选择一个起始顶点 u_0 ，首先将它加入到集合T中。
2. 在集合T中的顶点u与不在T中的顶点v之间选择权值最小的边(u, v)，将顶点v加入到集合T中。
3. 重复步骤2，直到网络中的所有顶点都加入到生成树顶点集合T中为止。

Kruskal算法的基本思想是从边的角度出发，逐步构建最小生成树。算法思想如下：

1. 连通网 $G = (V, E)$ ，令最小生成树的初始状态为只有n个顶点而无边的非连通图 $T = (V, \{\})$ ，即图中每个顶点自成一个连通分量。
2. 在E中选择权值最小的边，若该边依附的顶点落在T中不同的连通分量上（即不形成回路），则将此边加入到T中，否则舍去此边而选择下一条权值最小的边。
3. 重复上述步骤，直到T中所有顶点都在同一连通分量上为止。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

在Prim算法中用closedge数组标记各顶点到当前生成树的最短距离，其结构体定义如下：

```
typedef struct closedge{
    int adjvex;
    int lowcost;
}closedge;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
int MiniSpanTree_PRIM(MGraph G)
    //从顶点1出发用Prim算法构建最小生成树
    //返回树各边的长度之和
int MiniSpanTree_KRUS(MGraph G)
    //从顶点1出发用kruskal算法构建最小生成树
    //返回树各边的长度之和
```

调试分析

测试数据采用输入样例数据：

```
4 5
1 2 2
1 3 2
1 4 3
2 3 4
3 4 3
```

未发现明显问题。

算法的时空分析

各函数时空复杂度如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度：O(arcnum)
    //空间复杂度：O(1)
int MiniSpanTree_PRIM(MGraph G)
    //从顶点1出发用Prim算法构建最小生成树
    //返回树各边的长度之和
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(vexnum)
int MiniSpanTree_KRUS(MGraph G)
    //从顶点1出发用kruskal算法构建最小生成树
    //返回树各边的长度之和
    //时间复杂度：O(arcnum*log(arcnum))
    //空间复杂度：O(vexnum)
```

测试结果及分析

键盘输入：

```
4 5
1 2 2
1 3 2
1 4 3
2 3 4
3 4 3
```

显示器输出：

铁路交通网的最短路径

算法设计

Dijkstra算法通过贪心策略，不断选择当前路径代价最小的节点进行更新，逐步扩展搜索范围，直到找到从源节点到所有可达节点的最短路径。算法描述如下：

1. 从起点开始，将起点的距离设置为0，其他节点的距离设置为无穷大，表示还未找到到达该节点的路径。
2. 从未访问的节点中，选择距离起点最近的节点，然后将其标记为已访问。
3. 对选中节点的所有邻居节点，检查从起点经过当前节点到达邻居节点的路径是否比当前已知最短路径更短，如果是，则更新邻居节点的最短路径值。
4. 重复步骤2和步骤3，直到所有节点都被访问，或剩余未访问节点与起点不连通。
5. 算法完成时，每个节点的最短路径长度都已确定。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

用closedge数组标记各顶点到当前连通分量的最短距离，其结构体定义如下：

```
typedef struct closedge{
    int adjvex;
    int lowcost;
}closedge;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
int ShortestPath_DIJ(MGraph G, int start, int end)
    //用Dijkstra算法计算start到end的最短路径
    //返回最短路径的长度
```

调试分析

测试数据采用输入样例数据：

```
4 5
1 2 10
1 3 20
2 3 15
2 4 30
3 4 20
1 4
```

未发现明显问题。

算法的时空分析

各函数时空复杂度如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度：O(arcnum)
    //空间复杂度：O(1)
int ShortestPath_DIJ(MGraph G,int start,int end)
    //用Dijkstra算法计算start到end的最短路径
    //返回最短路径的长度
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(vexnum)
```

测试结果及分析

键盘输入：

```
4 5
1 2 10
1 3 20
2 3 15
2 4 30
3 4 20
1 4
```

显示器输出：

```
40
```

显示图

算法设计

希望边的交叉尽量少，采用力导向算法。假设有边的顶点之间存在弹簧弹力，满足 $F = K_1(r - r_0)$ ；每个顶点直接存在库仑斥力，满足 $F = \frac{K_2}{r^2}$ ，运动顶点受到阻尼力，满足 $F = -K_3v$ 。将 K_1, K_2, K_3, r_0 均视为手动调整的超参数，初始时将顶点均匀安排在圆周上，用受力模拟调整各顶点位置。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

矢量结构体定义如下：

```
typedef struct Vector{
    double x;
    double y;
}Vector;
```

各顶点的坐标与速度、加速度信息定义如下：

```
typedef struct Vertex{
    double x;
    double y;
    Vector v;
    Vector a;
}Vertex;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
void GetA(MGraph G,Vertex *ver)
    //计算各顶点的加速度
void GetNextStatus(MGraph G,Vertex *ver)
    //计算下一时刻的速度与坐标
void PrintGraph(MGraph G,Vertex *ver)
    //输出图
```

调试分析

测试数据采用输入样例数据：

```
5 6
1 5
1 4
3 1
2 1
2 5
5 3
1
```

需要多次手动调整各超参数。最终确定：


```
#define T_0 0.00001 //间隔时间
#define x_0 100 //弹簧0形变距离
#define K_1 0.5
#define K_2 0.00001
#define K_3 0.01
```

算法的时空分析

各函数时空复杂度如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度：O(arcnum)
    //空间复杂度：O(1)
void GetA(MGraph G,Vertex *ver)
    //计算各顶点的加速度
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(1)
void GetNextStatus(MGraph G,Vertex *ver)
    //计算下一时刻的速度与坐标
    //时间复杂度：O(vexnum)
    //空间复杂度：O(1)
void PrintGraph(MGraph G,Vertex *ver)
    //输出图
    //时间复杂度：O(vexnum+arcnum)
    //空间复杂度：O(1)
```

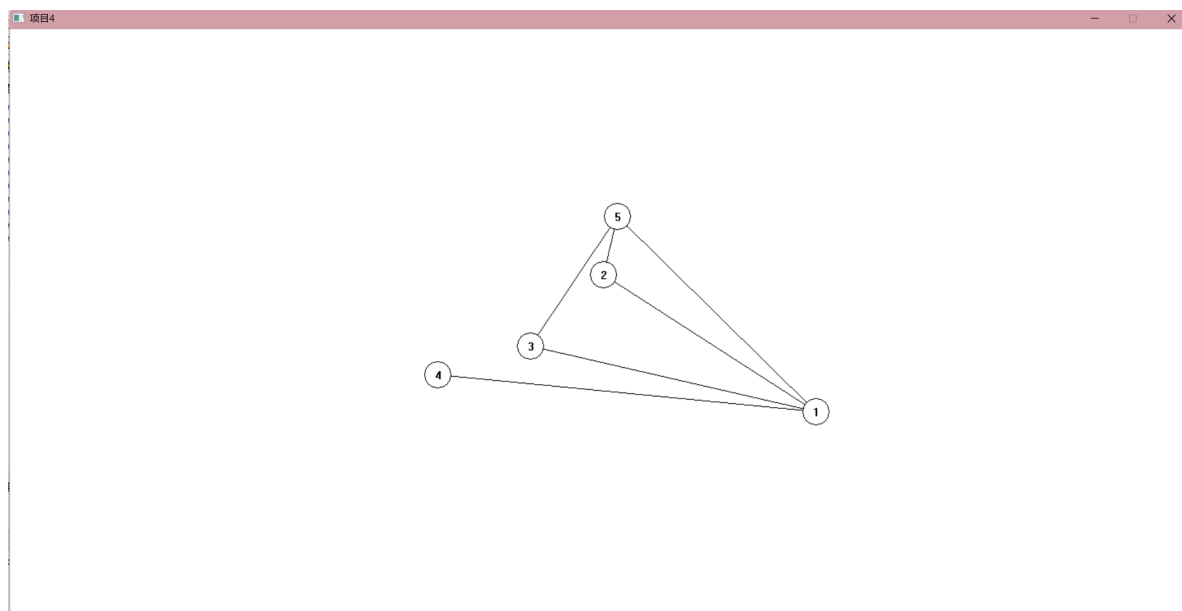
测试结果及分析

键盘输入：

```
5 6
1 5
1 4
3 1
2 1
2 5
5 3
```

显示器输出：

```
1 262.621448 106.274993
2 3.501857 -61.804436
3 -86.609065 26.129811
4 -199.993378 61.408305
5 20.479120 -132.008660 //各顶点坐标
```



实验体会与收获

通过本实验，我掌握了图的两种存储结构：邻接矩阵表示法和邻接表表示法；掌握了图的DFS遍历和BFS遍历的算法；学会了利用图的模型来编程解决实际问题。